

MapReduce con Ray.io per il conteggio delle parole in LOTR

Alessia Smeralda Brunello

Matricola: 544964

Hands-On 18

Indice

1	Introduzione al problema	2
2	Stato dell'arte	2
3	Metodologia usata	3
3.1	Pre-processing del testo	3
3.2	Fase Map	3
3.3	Fase Reduce	4
3.4	Automazione tramite Makefile	4
4	Risultati sperimentali	4
5	Conclusione e sviluppi futuri	6

1 Introduzione al problema

L'obiettivo dell'Hands-On 18 consiste nell'implementare un sistema distribuito basato sul modello *MapReduce*, utilizzando Python e Ray.io, per contare le occorrenze dei termini presenti nel testo di *The Lord of the Rings*.

Il risultato dell'elaborazione viene salvato in un file testuale contenente le parole più frequenti e il relativo numero di occorrenze.

Il problema affrontato è un classico esempio di *word count*: dato un testo di grandi dimensioni, si vuole determinare quante volte compare ogni parola.

Sebbene il conteggio delle parole sia semplice dal punto di vista logico, esso rappresenta un buon caso di studio per comprendere i principi della programmazione distribuita, poiché il testo può essere diviso in parti indipendenti ed elaborato in parallelo.

2 Stato dell'arte

MapReduce è un modello di programmazione pensato per elaborare grandi quantità di dati tramite due fasi principali: *Map* e *Reduce*. Il modello è stato reso noto da Google come soluzione per semplificare l'elaborazione distribuita di grandi dataset su cluster di macchine.

In generale, il funzionamento è il seguente:

- **Map**: i dati di input vengono suddivisi in parti più piccole e ogni parte viene elaborata separatamente;
- **Shuffle/Aggregazione**: i risultati intermedi vengono organizzati in base alla chiave;
- **Reduce**: i risultati parziali vengono combinati per ottenere il risultato finale.

Nel caso del conteggio delle parole:

- la fase **Map** conta le parole presenti in una porzione del testo
- la fase **Reduce** somma i conteggi parziali ottenuti dai diversi chunk.

Ray.io è un framework Python per lo sviluppo di applicazioni parallele e distribuite. Nel progetto è stato usato Ray Core, che permette di trasformare una normale funzione Python in un task remoto tramite il decoratore `@ray.remote`.

Una funzione remota può essere eseguita in parallelo tramite il metodo `.remote()`, mentre il risultato viene recuperato con `ray.get()`.

Rispetto a un'esecuzione sequenziale, l'utilizzo di Ray permette di distribuire il lavoro su più task, sfruttando meglio le risorse disponibili della macchina locale o, in scenari più avanzati, di un cluster.

3 Metodologia usata

Il progetto è stato organizzato in modo modulare, mantenendo una struttura semplice e utilizzando un **Makefile** per automatizzare l'installazione delle dipendenze e l'esecuzione.

3.1 Pre-processing del testo

Il testo di input è contenuto nel file `lotr.txt`. Prima del conteggio, il programma esegue alcune operazioni di pre-processing:

1. lettura del file di testo;
2. divisione del testo in blocchi, chiamati *chunk*;
3. estrazione delle parole tramite espressione regolare;
4. conversione delle parole in minuscolo;
5. rimozione opzionale delle stopwords;
6. rimozione opzionale delle parole più corte di una lunghezza minima.

La tokenizzazione è stata realizzata con una regex compatibile con caratteri Unicode, in modo da riconoscere anche parole contenenti lettere accentate, come alcuni nomi propri presenti nel testo.

3.2 Fase Map

La fase Map è implementata in `mapreduce.py` nella funzione `map_task`, decorata con `@ray.remote`.

Ogni task riceve un chunk di testo e restituisce un dizionario contenente il conteggio locale delle parole.

Il risultato di un singolo task Map ha una struttura del tipo:

```
{  
    "frodo": 12,  
    "ring": 8,  
    "gandalf": 5  
}
```

Ogni chunk può essere elaborato indipendentemente dagli altri, quindi questa fase si presta bene all'esecuzione parallela.

3.3 Fase Reduce

La fase Reduce è implementata sempre in `mapreduce.py` nella funzione `reduce_task`, anch'essa decorata con `@ray.remote`. Questa funzione riceve una lista di dizionari parziali e li aggrega sommando le occorrenze delle stesse parole.

Ad esempio:

Input:

```
[
    {"ring": 2, "frodo": 1},
    {"ring": 3, "gandalf": 1}
]
```

Output:

```
{
    "ring": 5,
    "frodo": 1,
    "gandalf": 1
}
```

Nel progetto, la riduzione viene eseguita a gruppi, usando il parametro `reduce_batch_size`. In questo modo non tutti i risultati parziali vengono aggregati in un'unica operazione, ma vengono combinati progressivamente fino a ottenere un solo risultato finale.

3.4 Automazione tramite Makefile

Il `Makefile` consente di eseguire il progetto in modo semplice. Il comando principale è:

```
make run
```

Alcuni parametri possono essere modificati direttamente da terminale, ad esempio:

```
make run TOP=N
make run MIN_LEN=N
make run STOPWORDS=stopwords_en.txt
```

Questa scelta rende il progetto più flessibile, perché permette di modificare il numero di parole salvate, la lunghezza minima dei termini o l'uso delle stopwords senza cambiare il codice sorgente.

4 Risultati sperimentali

L'esecuzione del programma sul file `lotr.txt` ha prodotto il file `word_count.txt`.

Nel test effettuato, il testo è stato suddiviso in 52 chunk, ciascuno di dimensione approssimativa pari a 20000 caratteri.

Eseguito il programma senza stopwords, le parole più frequenti sono risultate essere soprattutto articoli, preposizioni e pronomi della lingua inglese.

```
Parole totali contate: 188479
Parole distinte: 8848
File creato: word_count.txt

Top 20 parole:
the: 11738
and: 7564
of: 5085
to: 3950
a: 3729
he: 2991
in: 2964
it: 2720
i: 2709
that: 2497
was: 2461
they: 1974
you: 1798
but: 1771
his: 1574
said: 1481
not: 1450
for: 1448
as: 1394
is: 1342
```

Successivamente, è stata eseguita anche una versione con stopwords, cioè rimuovendo alcune parole comuni della lingua inglese. In questo caso emergono termini più significativi rispetto al contenuto del testo.

```
Parole totali contate: 105467
Parole distinte: 8797
File creato: word_count.txt

Top 20 parole:
said: 1481
frodo: 1037
now: 680
no: 649
out: 636
up: 604
will: 563
do: 513
into: 460
long: 457
been: 455
could: 455
gandalf: 448
would: 443
came: 436
down: 427
again: 418
more: 415
like: 411
before: 397
```

Dal confronto tra i due risultati si nota che:

- senza stopwords il conteggio è completo, ma le parole più frequenti sono spesso poco informative;
- con stopwords il risultato è più utile per osservare termini legati al contenuto del testo;
- il modello MapReduce consente di separare chiaramente il lavoro in sotto-problemi indipendenti e aggregazione finale.

5 Conclusione e sviluppi futuri

Il progetto ha permesso di implementare un sistema MapReduce distribuito in Python utilizzando Ray.io.

Il problema del conteggio delle parole è stato suddiviso in più fasi:

- lettura del testo,
- suddivisione in chunk,
- tokenizzazione,
- fase Map,
- fase Reduce,
- e scrittura dei risultati finali.

Tra i possibili sviluppi futuri, si potrebbero considerare:

- esecuzione del programma su un vero cluster Ray distribuito;
- confronto dei tempi di esecuzione tra versione sequenziale e versione parallela;
- generazione automatica di grafici sulle parole più frequenti;
- analisi separata dei diversi libri o capitoli;
- salvataggio dei risultati anche in formato CSV o JSON.

In conclusione, il progetto mostra come un problema semplice come il word count possa essere usato per comprendere i concetti fondamentali della programmazione distribuita: divisione del lavoro, parallelismo, aggregazione dei risultati e automazione dell'esecuzione.